

then the folder remains in its original location on disk and it becomes an item in the [StorageLibrary.Folders](#) property (and in the built-in Photos app), but the folder does not appear as a child of the Pictures folder in File Explorer.

C#

```
Windows.Storage.StorageFolder newFolder = await  
myPictures.RequestAddFolderAsync();
```

Remove a folder from a library

To remove a folder from a library, call the [StorageLibrary.RequestRemoveFolderAsync](#) method and specify the folder to be removed. You could use [StorageLibrary.Folders](#) and a [ListView](#) control (or similar) for the user to select a folder to remove.

When you call [StorageLibrary.RequestRemoveFolderAsync](#), the user sees a confirmation dialog saying that the folder "won't appear in Pictures anymore, but won't be deleted." What this means is that the folder remains in its original location on disk, is removed from the [StorageLibrary.Folders](#) property, and will no longer be included in the built-in Photos app.

The following example assumes that the user has selected the folder to remove from a [ListView](#) control named `lvPictureFolders`.

C#

```
bool result = await myPictures.RequestRemoveFolderAsync(folder);
```

Get notified of changes to the list of folders in a library

To get notified about changes to the list of folders in a library, register a handler for the [StorageLibrary.DefinitionChanged](#) event of the library.

C#

```
myPictures.DefinitionChanged += MyPictures_DefinitionChanged;  
  
void HandleDefinitionChanged(Windows.Storage.StorageLibrary sender, object  
args)  
{  
    // ...  
}
```

Media library folders

A device provides five predefined locations for users and apps to store media files. Built-in apps store both user-created media and downloaded media in these locations.

The locations are:

- **Pictures** folder. Contains pictures.
 - **Camera Roll** folder. Contains photos and video from the built-in camera.
 - **Saved Pictures** folder. Contains pictures that the user has saved from other apps.
- **Music** folder. Contains songs, podcasts, and audio books.
- **Video** folder. Contains videos.

Users or apps may also store media files outside the media library folders on the SD card. To find a media file reliably on the SD card, scan the contents of the SD card, or ask the user to locate the file by using a file picker. For more info, see [Access the SD card](#).

Querying the media libraries

To get a collection of files, specify the library and the type of files that you want.

C#

```
using Windows.Storage;
using Windows.Storage.Search;

private async void getSongs()
{
    QueryOptions queryOption = new QueryOptions
        (CommonFileQuery.OrderByTitle, new string[] { ".mp3", ".mp4", ".wma"
    });

    queryOption.FolderDepth = FolderDepth.Deep;

    Queue<IStorageFolder> folders = new Queue<IStorageFolder>();

    var files = await KnownFolders.MusicLibrary.CreateFileQueryWithOptions
        (queryOption).GetFilesAsync();

    foreach (var file in files)
    {
```

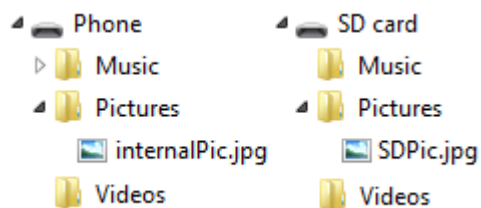
```
    // do something with the music files
  }
}
```

Query results include both internal and removable storage

Users can choose to store files by default on the optional SD card. Apps, however, can opt out of allowing files to be stored on the SD card. As a result, the media libraries can be split across the device's internal storage and the SD card.

You don't have to write additional code to handle this possibility. The methods in the [Windows.Storage](#) namespace that query known folders transparently combine the query results from both locations. You don't have to specify the **removableStorage** capability in the app manifest file to get these combined results, either.

Consider the state of the device's storage shown in the following image:



If you query the contents of the Pictures Library by calling `await KnownFolders.PicturesLibrary.GetFilesAsync()`, the results include both `internalPic.jpg` and `SDPic.jpg`.

Working with photos

On devices where the camera saves both a low-resolution image and a high-resolution image of every picture, the deep queries return only the low-resolution image.

The Camera Roll and the Saved Pictures folder do not support the deep queries.

Opening a photo in the app that captured it

If you want to let the user open a photo again later in the app that captured it, you can save the **CreatorAppId** with the photo's metadata by using code similar to the following example. In this example, `testPhoto` is a [StorageFile](#).

C#

```

IDictionary<string, object> propertiesToSave = new Dictionary<string,
object>();

propertiesToSave.Add("System.CreatorOpenWithUIOptions", 1);
propertiesToSave.Add("System.CreatorAppId", appId);

testPhoto.Properties.SavePropertiesAsync(propertiesToSave).Await();

```

Using stream methods to add a file to a media library

When you access a media library by using a known folder such as **KnownFolders.PictureLibrary**, and you use stream methods to add a file to the media library, you have to make sure to close all the streams that your code opens. Otherwise these methods fail to add the file to the media library as expected because at least one stream still has a handle to the file.

For example, when you run the following code, the file is not added to the media library. In the line of code, `using (var destinationStream = (await destinationFile.OpenAsync(FileAccessMode.ReadWrite)).GetOutputStreamAt(0))`, both the **OpenAsync** method and the **GetOutputStreamAt** method open a stream. However only the stream opened by the **GetOutputStreamAt** method is disposed as a result of the **using** statement. The other stream remains open and prevents saving the file.

C#

```

StorageFolder testFolder = await
StorageFolder.GetFolderFromPathAsync(@"C:\test");
StorageFile sourceFile = await testFolder.GetFilesAsync("TestImage.jpg");
StorageFile destinationFile = await
KnownFolders.CameraRoll.CreateFileAsync("MyTestImage.jpg");
using (var sourceStream = (await
sourceFile.OpenReadAsync()).GetInputStreamAt(0))
{
    using (var destinationStream = (await
destinationFile.OpenAsync(FileAccessMode.ReadWrite)).GetOutputStreamAt(0))
    {
        await RandomAccessStream.CopyAndCloseAsync(sourceStream,
destinationStream);
    }
}

```

To use stream methods successfully to add a file to the media library, make sure to close all the streams that your code opens, as shown in the following example.

C#

```
StorageFolder testFolder = await
StorageFolder.GetFolderFromPathAsync(@"C:\test");
StorageFile sourceFile = await testFolder.GetFilesAsync("TestImage.jpg");
StorageFile destinationFile = await
KnownFolders.CameraRoll.CreateFileAsync("MyTestImage.jpg");

using (var sourceStream = await sourceFile.OpenReadAsync())
{
    using (var sourceInputStream = sourceStream.GetInputStreamAt(0))
    {
        using (var destinationStream = await
destinationFile.OpenAsync(FileAccessMode.ReadWrite))
        {
            using (var destinationOutputStream =
destinationStream.GetOutputStreamAt(0))
            {
                await
RandomAccessStream.CopyAndCloseAsync(sourceInputStream, destinationStream);
            }
        }
    }
}
```

Thumbnail images

Article • 10/20/2022

These guidelines describe how to use thumbnail images to help users preview files as they browse in your UWP app.

Important APIs

- [ThumbnailMode](#)

Should my app include thumbnails?

If your app allows users to browse files, you can display thumbnail images to help users quickly preview those files.

Use thumbnails when:

- Displaying previews for many items in a gallery collection (like files and folders). For example, a photo gallery should use thumbnails to give users a small view of each picture as they browse through their photo files.

Episodes



Interview with Beth Pan Software Engineer on the MVA Team

Community women in tech

9/25/2017 2:00:00 PM +00:00 00:28:05



Interview With Vidya Srinivasan, Engineer Singer and Speaker

Community Communities GALs women in tech

9/12/2017 1:03:21 AM +00:00 00:32:31



Meet Lili Cheng, Vice President, Artificial Intelligence and Research

Community Communities GALs women in tech

9/8/2017 3:06:10 PM +00:00 00:31:00



Interview with Irina Frumkin, Software Engineering Manager at Microsoft

Community Communities

8/28/2017 10:24:20 PM +00:00 00:29:00



Interview with Maira Wenzel, Senior Content Developer on the .NET Team

Community women in tech

8/21/2017 2:00:00 PM +00:00 00:20:54

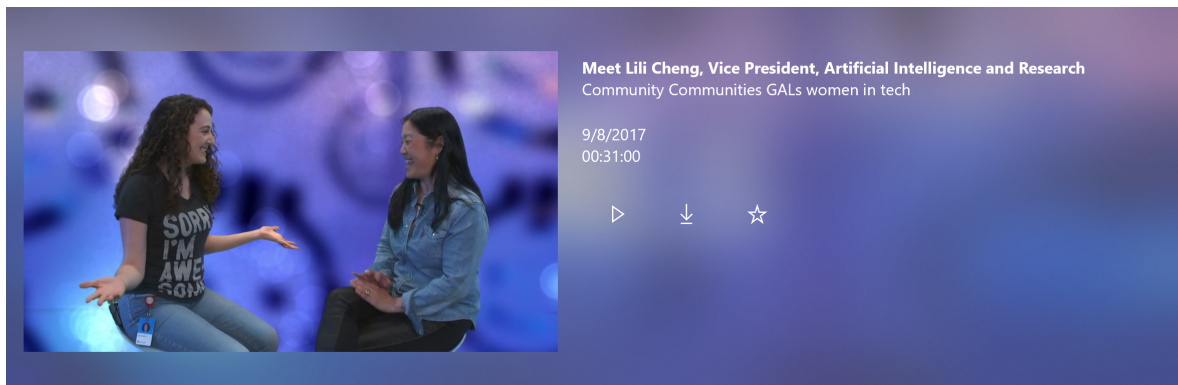


Interview with Ginger Reinauer, UX designer at Expeditors

Community women in tech

8/7/2017 2:30:00 PM +00:00 00:25:35

- Displaying a preview for an individual item in a list (like a specific file). For example, the user may want to see more information about a file, including a larger thumbnail for a better preview, before deciding whether to open the file.



Dos and don'ts

- Specify the [thumbnail mode](#) (PicturesView, VideosView, DocumentsView, MusicView, ListView, or SingleItem) when you retrieve thumbnails. This ensures that thumbnail images are optimized to display the type of files users want to see.
 - Use the SingleItem mode to retrieve a thumbnail for a single item, regardless of file type. The other thumbnail modes are meant to display previews of multiple files.
- Display generic placeholder images in place of thumbnails while thumbnails load. Using placeholders helps your app seem more responsive because users can interact with previews before the thumbnail load.

Placeholder images should be:

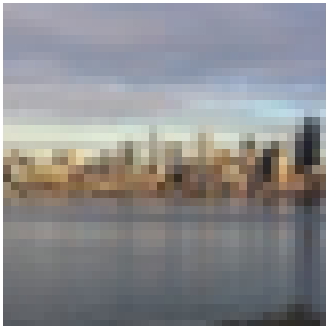
- Specific to the kind of item that it stands in for. For example, folders, pictures, and videos should all have their own specialized placeholders.
 - The same size and aspect ratio as the thumbnail image it stands in for.
 - Displayed until the thumbnail image is loaded.
- Use placeholder images with text labels to represent folders and file groups to differentiate from individual files.
 - If you can't retrieve a thumbnail, display a placeholder image.
 - Display additional file information when providing previews for document and music files. Users can then identify key information about a file that might not be readily available from a thumbnail image alone. For example, for a music file, you might display the name of the artist along with the thumbnail of the album art.
 - Don't display additional file info for picture and video files. In most cases, a thumbnail image is sufficient for users browsing through pictures and videos.

Additional usage guidelines

Recommended [thumbnail modes](#) and their features:

Display previews for	Thumbnail modes	Features of the retrieved thumbnail images
Pictures Videos	PicturesView VideosView	Size: Medium, preferably at least 190 (if the image size is 190x130) Aspect ratio: Uniform, wide aspect ratio of about .7 (190x130 if the size is 190) Cropped for previews. Good for aligning images in a grid because of uniform aspect ratio.
Documents Music	DocumentsView MusicView ListView	Size: Small, preferably at least 40 x 40 pixels Aspect ratio: Uniform, square aspect ratio Good for previewing album art because of the square aspect ratio. Documents look the same as they look in a file picker window (it uses the same icons).
Any single item (regardless of file type)	SingleItem	Size: Small, preferably at least 40 x 40 pixels Aspect ratio: Uniform, square aspect ratio Good for previewing album art because of the square aspect ratio. Documents look the same as they look in a file picker window (it uses the same icons).

Here are examples showing how retrieved thumbnail images differ depending on file type and thumbnail mode:

Item type	When retrieved using:	When retrieved using:	When retrieved using:
	• PicturesView • VideosView	• DocumentsView • MusicView • ListView	• SingleItem
Picture	The thumbnail image uses a uniform, wide aspect ratio of about .7 (190 x 130 if the preferred	The thumbnail is cropped to a square aspect ratio. 	The thumbnail image uses the original aspect ratio of the file. 

size is 190).

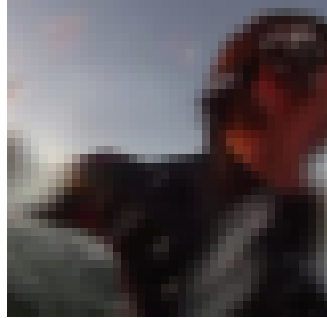


Video

The thumbnail has an icon that differentiates it from pictures.



The thumbnail is cropped to a square aspect ratio.

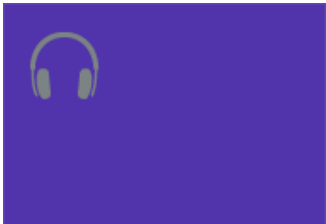


The thumbnail image uses the original aspect ratio of the file.



Music

The thumbnail is an icon on a background of appropriate size. The background color is determined by the app's tile background color.



If the file has album art, then the thumbnail is the album art.



Otherwise, the thumbnail is an icon on a background of appropriate size.

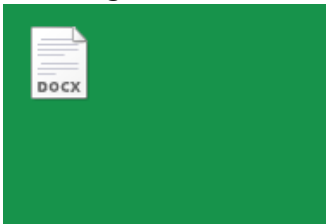
If the file has album art, then the thumbnail is the album art with the original aspect ratio of the file.



Otherwise, the thumbnail is an icon.

Document

The thumbnail is an icon on a background of appropriate size. The background color is determined by the app's tile background color.



The thumbnail is an icon on a background of appropriate size. The background color is determined by the app's tile background color.



The document thumbnail, if one exists.



Otherwise, the thumbnail is

an icon.



Folder

If there is a picture file in the folder, then the picture thumbnail is used.



Otherwise, no thumbnail is retrieved.

No thumbnail image is retrieved.

The thumbnail is the folder icon.



File group

If there is a picture file in the folder, then the picture thumbnail is used.



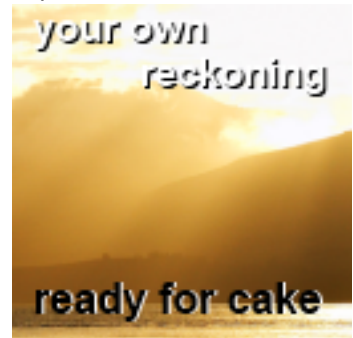
Otherwise, no thumbnail is retrieved.

If there is a file that has album art among the files in the group, the thumbnail is the album art.



Otherwise, no thumbnail is retrieved.

If there is a file that has album art among the files in the group, the thumbnail is the album art and uses the original aspect ratio of the file.



Otherwise, the thumbnail is an icon that represents a group of files.



Related topics

- [ThumbnailMode enum](#)
- [StorageItemThumbnail class](#)
- [StorageFile class](#)
- [File and folder thumbnail sample \(GitHub\) ↗](#)
- [List and grid view](#)

Track recently used files and folders

Article • 10/20/2022

Important APIs

- [MostRecentlyUsedList](#)
- [FileOpenPicker](#)

Track files that your user accesses frequently by adding them to your app's most recently used list (MRU). The platform manages the MRU for you by sorting items based on when they were last accessed, and by removing the oldest item when the list's 25-item limit is reached. All apps have their own MRU.

Your app's MRU is represented by the [StorageItemMostRecentlyUsedList](#) class, which you obtain from the static [StorageApplicationPermissions.MostRecentlyUsedList](#) property. MRU items are stored as [IStorageItem](#) objects, so both [StorageFile](#) objects (which represent files) and [StorageFolder](#) objects (which represent folders) can be added to the MRU.

ⓘ Note

For complete samples, see the [File picker sample](#) [↗](#) and the [File access sample](#) [↗](#).

Prerequisites

- Understand async programming for Universal Windows Platform (UWP) apps

You can learn how to write asynchronous apps in C# or Visual Basic, see [Call asynchronous APIs in C# or Visual Basic](#). To learn how to write asynchronous apps in C++, see [Asynchronous programming in C++](#).

- Access permissions to the location

See [File access permissions](#).

- [Open files and folders with a picker](#)

Picked files are often the same files that users return to again and again.

Add a picked file to the MRU

- The files that your user picks are often files that they return to repeatedly. So consider adding picked files to your app's MRU as soon as they are picked. Here's how.

```
C#

Windows.Storage.StorageFile file = await picker.PickSingleFileAsync();

var mru =
Windows.Storage.AccessCache.StorageApplicationPermissions.MostRecentlyUsedList;
string mruToken = mru.Add(file, "profile pic");
```

[StorageItemMostRecentlyUsedList.Add](#) is overloaded. In the example, we use [Add\(IStorageItem, String\)](#) so that we can associate metadata with the file. Setting metadata lets you record the item's purpose, for example "profile pic". You can also add the file to the MRU without metadata by calling [Add\(IStorageItem\)](#). When you add an item to the MRU, the method returns a uniquely identifying string, called a token, which is used to retrieve the item.

Tip

You'll need the token to retrieve an item from the MRU, so persist it somewhere. For more info about app data, see [Managing application data](#).

Use a token to retrieve an item from the MRU

Use the retrieval method most appropriate for the item you want to retrieve.

- Retrieve a file as a [StorageFile](#) by using [GetFileAsync](#).
- Retrieve a folder as a [StorageFolder](#) by using [GetFolderAsync](#).
- Retrieve a generic [IStorageItem](#), which can represent either a file or folder, by using [GetItemAsync](#).

Here's how to get back the file we just added.

```
C#

StorageFile retrievedFile = await mru.GetFileAsync(mruToken);
```

Here's how to iterate all the entries to get tokens and then items.

```
C#
```

```
foreach (Windows.Storage.AccessCache.AccessListEntry entry in mru.Entries)
{
    string mruToken = entry.Token;
    string mruMetadata = entry.Metadata;
    Windows.Storage.IStorageItem item = await mru.GetItemAsync(mruToken);
    // The type of item will tell you whether it's a file or a folder.
}
```

The [AccessListView](#) lets you iterate entries in the MRU. These entries are [AccessListEntry](#) structures that contain the token and metadata for an item.

Removing items from the MRU when it's full

When the MRU's 25-item limit is reached and you try to add a new item, the item that was accessed the longest time ago is automatically removed. So, you never need to remove an item before you add a new one.

Future-access list

As well as an MRU, your app also has a future-access list. By picking files and folders, your user grants your app permission to access items that might not be accessible otherwise. If you add these items to your future-access list then you'll retain that permission when your app wants to access those items again later. Your app's future-access list is represented by the [StorageItemAccessList](#) class, which you obtain from the static [StorageApplicationPermissions.FutureAccessList](#) property.

When a user picks an item, consider adding it to your future-access list as well as your MRU.

- The [FutureAccessList](#) can hold up to 1000 items. Remember: it can hold folders as well as files, so that's a lot of folders.
- The platform never removes items from the [FutureAccessList](#) for you. When you reach the 1000-item limit, you can't add another until you make room with the [Remove](#) method.

Track file system changes in the background

Article • 10/20/2022

Important APIs

- [StorageLibraryChangeTracker](#)
- [StorageLibraryChangeReader](#)
- [StorageLibraryChangedTrigger](#)
- [StorageLibrary](#)

The [StorageLibraryChangeTracker](#) class allows apps to track changes in files and folders as users move them around the system. Using the [StorageLibraryChangeTracker](#) class, an app can track:

- File operations including add, delete, modify.
- Folder operations such as renames and deletes.
- Files and folders moving on the drive.

Use this guide to learn the programming model for working with the change tracker, view some sample code, and understand the different types of file operations that are tracked by [StorageLibraryChangeTracker](#).

[StorageLibraryChangeTracker](#) works for user libraries, or for any folder on the local machine. This includes secondary drives or removable drives but does not include NAS drives or network drives.

Using the change tracker

The change tracker is implemented on system as a circular buffer storing the last N file system operations. Apps are able to read the changes off of the buffer and then process them into their own experiences. Once the app is finished with the changes it marks the changes as processed and will never see them again.

To use the change tracker on a folder, follow these steps:

1. Enable change tracking for the folder.
2. Wait for changes.
3. Read changes.
4. Accept changes.

The next sections walk through each of the steps with some code examples. The complete code sample is provided at the end of the article.

Enable the change tracker

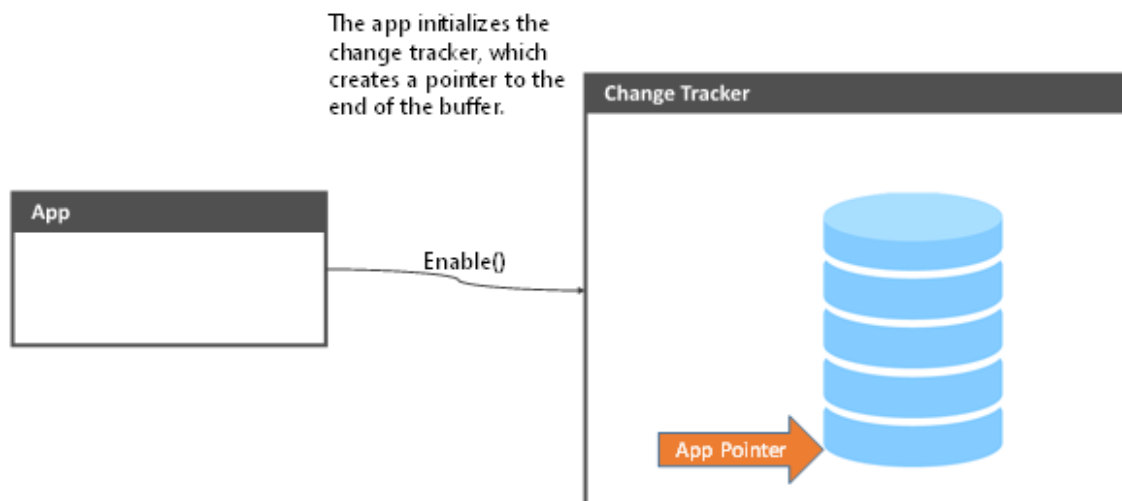
The first thing that the app needs to do is to tell the system that it is interested in change tracking a given library. It does this by calling the **Enable** method on the change tracker for the library of interest.

C#

```
StorageLibrary videosLib = await  
StorageLibrary.GetLibraryAsync(KnownLibraryId.Videos);  
StorageLibraryChangeTracker videoTracker = videosLib.ChangeTracker;  
videoTracker.Enable();
```

A few important notes:

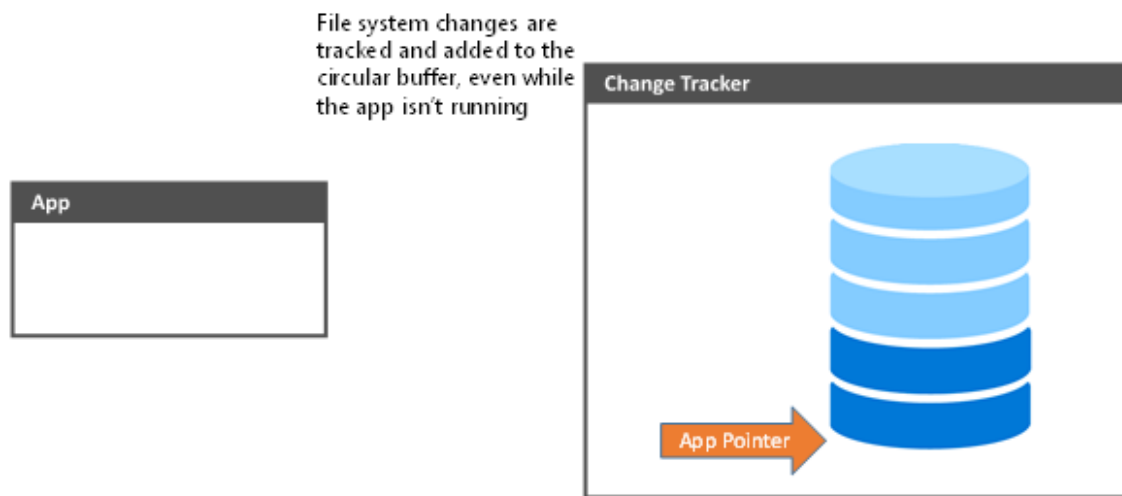
- Make sure your app has permission to the correct library in the manifest before creating the **StorageLibrary** object. See [File Access Permissions](#) for more details.
- **Enable** is thread safe, will not reset your pointer, and can be called as many times as you like (more on this later).



Wait for changes

After the change tracker is initialized, it will begin to record all of the operations that occur within a library, even while the app isn't running. Apps can register to be activated

any time there is a change by registering for the [StorageLibraryChangedTrigger](#) event.



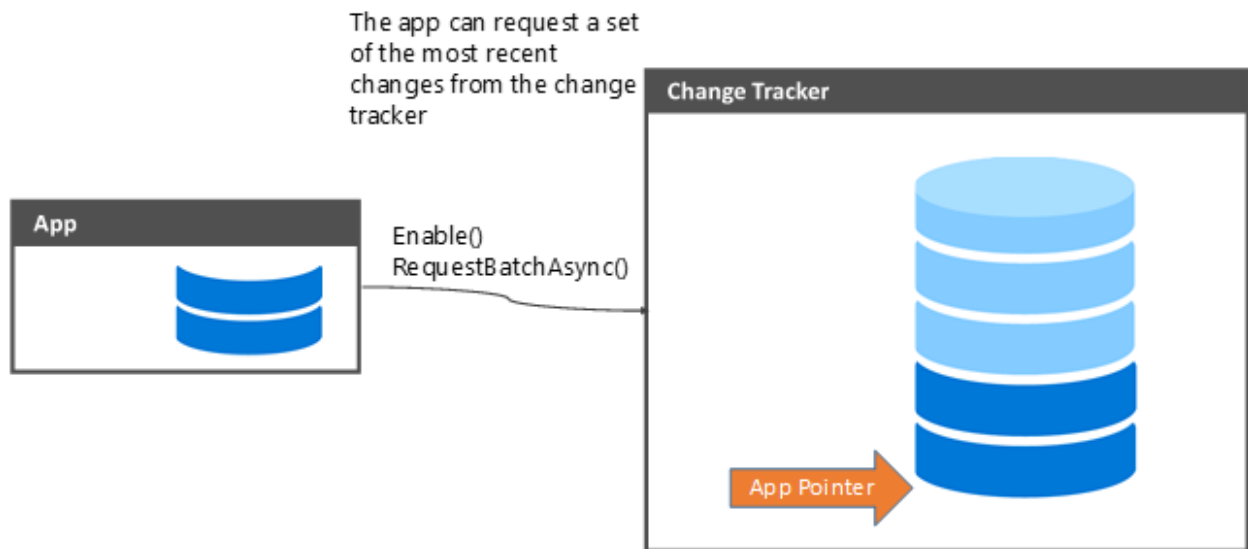
Read the changes

The app can then poll for changes from the change tracker and receive a list of the changes since the last time it checked. The code below shows how to get a list of changes from the change tracker.

C#

```
StorageLibrary videosLibrary = await
StorageLibrary.GetLibraryAsync(KnownLibraryId.Videos);
videosLibrary.ChangeTracker.Enable();
StorageLibraryChangeReader videoChangeReader =
videosLibrary.ChangeTracker.GetChangeReader();
IReadOnlyList changeSet = await changeReader.ReadBatchAsync();
```

The app is then responsible for processing the changes into its own experience or database as needed.



💡 Tip

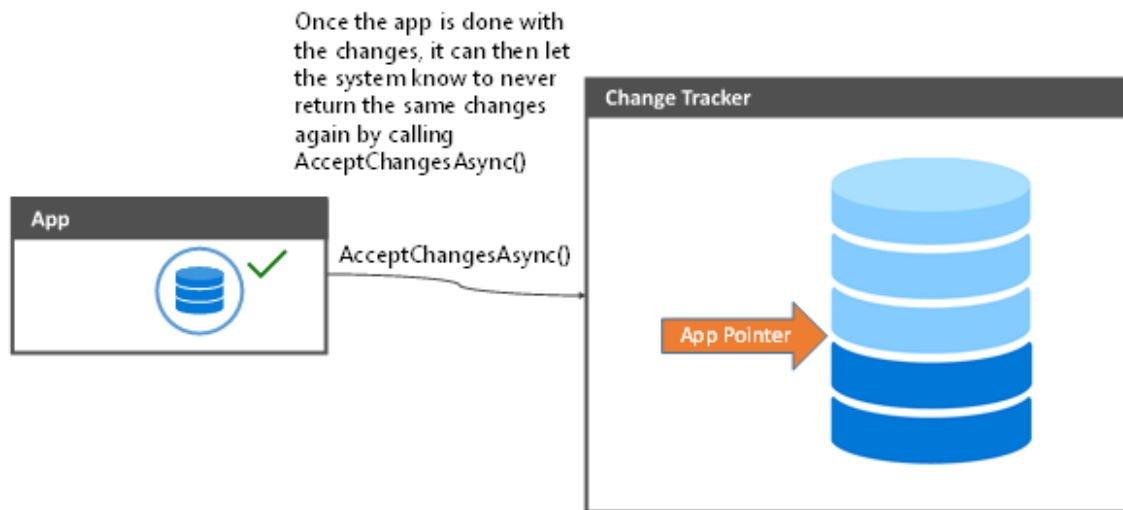
The second call to enable is to defend against a race condition if the user adds another folder to the library while your app is reading changes. Without the extra call to **Enable** the code will fail with `ecSearchFolderScopeViolation (0x80070490)` if the user is changing the folders in their library

Accept the changes

After the app is done processing the changes, it should tell the system to never show those changes again by calling the [AcceptChangesAsync](#) method.

C#

```
await changeReader.AcceptChangesAsync();
```



The app will now only receive new changes when reading the change tracker in the future.

- If changes have happened between calling `ReadBatchAsync` and `AcceptChangesAsync`, the pointer will only be advanced to the most recent change the app has seen. Those other changes will still be available the next time it calls `ReadBatchAsync`.
- Not accepting the changes will cause the system to return the same set of changes the next time the app calls `ReadBatchAsync`.

Important things to remember

When using the change tracker, there are a few things that you should keep in mind to make sure that everything is working correctly.

Buffer overruns

Although we try to reserve enough space in the change tracker to hold all the operations happening on the system until your app can read them, it is very easy to imagine a scenario where the app doesn't read the changes before the circular buffer overwrites itself. Especially if the user is restoring data from a backup or syncing a large collection of pictures from their camera phone.

In this case, `ReadBatchAsync` will return the error code `StorageLibraryChangeType.ChangeTrackingLost`. If your app receives this error code, it means a couple things:

- The buffer has overwritten itself since the last time you looked at it. The best course of action is to recrawl the library, because any information from the tracker will be incomplete.
- The change tracker will not return any more changes until you call [Reset](#). After the app calls reset, the pointer will be moved to the most recent change and tracking will resume normally.

It should be rare to get these cases, but in scenarios where the user is moving a large number of files around on their disk we don't want the change tracker to balloon and take up too much storage. This should allow apps to react to massive file system operations while not damaging the customer experience in Windows.

Changes to a StorageLibrary

The [StorageLibrary](#) class exists as a virtual group of root folders that contain other folders. To reconcile this with a file system change tracker, we made the following choices:

- Any changes to descendent of the root library folders will be represented in the change tracker. The root library folders can be found using the [Folders](#) property.
- Adding or removing root folders from a **StorageLibrary** (through [RequestAddFolderAsync](#) and [RequestRemoveFolderAsync](#)) will not create an entry in the change tracker. These changes can be tracked through the [DefinitionChanged](#) event or by enumerating the root folders in the library using the [Folders](#) property.
- If a folder with content already in it is added to the library, there will not be a change notification or change tracker entries generated. Any subsequent changes to the descendants of that folder will generate notifications and change tracker entries.

Calling the Enable method

Apps should call [Enable](#) as soon as they start tracking the file system and before every enumeration of the changes. This will ensure that all changes will be captured by the change tracker.

Putting it together

Here is all the code that is used to register for the changes from the video library and start pulling the changes from the change tracker.

C#

```
private async void EnableChangeTracker()
{
    StorageLibrary videosLib = await
StorageLibrary.GetLibraryAsync(KnownLibraryId.Videos);
    StorageLibraryChangeTracker videoTracker = videosLib.ChangeTracker;
    videoTracker.Enable();
}

private async void GetChanges()
{
    StorageLibrary videosLibrary = await
StorageLibrary.GetLibraryAsync(KnownLibraryId.Videos);
    videosLibrary.ChangeTracker.Enable();
    StorageLibraryChangeReader videoChangeReader =
videosLibrary.ChangeTracker.GetChangeReader();
    IReadOnlyList changeSet = await changeReader.ReadBatchAsync();

    //Below this line is for the blog post. Above the line is for the
magazine
    foreach (StorageLibraryChange change in changeSet)
    {
        if (change.ChangeType ==
StorageLibraryChangeType.ChangeTrackingLost)
        {
            //We are in trouble. Nothing else is going to be valid.
            log("Resetting the change tracker");
            videosLibrary.ChangeTracker.Reset();
            return;
        }
        if (change.IsOfType(StorageItemTypes.Folder))
        {
            await HandleFileChange(change);
        }
        else if (change.IsOfType(StorageItemTypes.File))
        {
            await HandleFolderChange(change);
        }
        else if (change.IsOfType(StorageItemTypes.None))
        {
            if (change.ChangeType == StorageLibraryChangeType.Deleted)
            {
                RemoveItemFromDB(change.Path);
            }
        }
    }
    await changeReader.AcceptChangesAsync();
}
```

Access the SD card

Article • 10/20/2022

You can store and access non-essential data on an optional microSD card, especially on low-cost mobile devices that have limited internal storage and have a slot for an SD card.

In most cases, you have to specify the **removableStorage** capability in the app manifest file before your app can store and access files on the SD card. Typically you also have to register to handle the type of files that your app stores and accesses.

You can store and access files on the optional SD card by using the following methods:

- File pickers.
- The [Windows.Storage](#) APIs.

What you can and can't access on the SD card

What you can access

- Your app can only read and write files of file types that the app has registered to handle in the app manifest file.
- Your app can also create and manage folders.

What you can't access

- Your app can't see or access system folders and the files that they contain.
- Your app can't see files that are marked with the Hidden attribute. The Hidden attribute is typically used to reduce the risk of deleting data accidentally.
- Your app can't see or access the Documents library by using [KnownFolders.DocumentsLibrary](#). However you can access the Documents library on the SD card by traversing the file system.

Security and privacy considerations

When an app saves files in a global location on the SD card, those files are not encrypted so they are typically accessible to other apps.

- While the SD card is in the device, your files are accessible to other apps that have registered to handle the same file type.

- When the SD card is removed from the device and opened from a PC, your files are visible in File Explorer and accessible to other apps.

When an app installed on the SD card saves files in its [LocalFolder](#), however, those files are encrypted and are not accessible to other apps.

Requirements for accessing files on the SD card

To access files on the SD card, typically you have to specify the following things.

1. You have to specify the **removableStorage** capability in the app manifest file.
2. You also have to register to handle the file extensions associated with the type of media that you want to access.

Use the preceding method also to access media files on the SD card without referencing a known folder like **KnownFolders.MusicLibrary**, or to access media files that are stored outside of the media library folders.

To access media files stored in the media libraries—Music, Photos, or Videos—by using known folders, you only have to specify the associated capability in the app manifest file—**musicLibrary**, **picturesLibrary**, or **videoLibrary**. You do not have to specify the **removableStorage** capability. For more info, see [Files and folders in the Music, Pictures, and Videos libraries](#).

Accessing files on the SD card

Getting a reference to the SD card

The [KnownFolders.RemovableDevices](#) folder is the logical root [StorageFolder](#) for the set of removable devices currently connected to the device. If an SD card is present, the first (and only) **StorageFolder** underneath the **KnownFolders.RemovableDevices** folder represents the SD card.

Use code like the following to determine whether an SD card is present and to get a reference to it as a [StorageFolder](#).

C#

```
using Windows.Storage;

// Get the logical root folder for all external storage devices.
StorageFolder externalDevices =
Windows.Storage.KnownFolders.RemovableDevices;
```

```
// Get the first child folder, which represents the SD card.
StorageFolder sdCard = (await
externalDevices.GetFoldersAsync()).FirstOrDefault();

if (sdCard != null)
{
    // An SD card is present and the sdCard variable now contains a
    reference to it.
}
else
{
    // No SD card is present.
}
```

⚠ Note

If your SD card reader is an embedded reader (for example, a slot in the laptop or PC itself), it may not be accessible through `KnownFolders.RemovableDevices`.

Querying the contents of the SD card

The SD card can contain many folders and files that aren't recognized as known folders and can't be queried by using a location from [KnownFolders](#). To find files, your app has to enumerate the contents of the card by traversing the file system recursively. Use [GetFilesAsync \(CommonFileQuery.DefaultQuery\)](#) and [GetFoldersAsync \(CommonFolderQuery.DefaultQuery\)](#) to get the contents of the SD card efficiently.

We recommend that you use a background thread to traverse the SD card. An SD card may contain many gigabytes of data.

Your app can also require the user to choose specific folders by using the folder picker.

When you access the file system on the SD card with a path that you derived from [KnownFolders.RemovableDevices](#), the following methods behave in the following way.

- The [GetFilesAsync](#) method returns the union of the file extensions that you have registered to handle and the file extensions associated with any media library capabilities that you have specified.
- The [GetFileFromPathAsync](#) method fails if you have not registered to handle the file extension of the file you are trying to access.

Identifying the individual SD card

When the SD card is first mounted, the operating system generates a unique identifier for the card. It stores this ID in a file in the WPSYSTEM folder at the root of the card. An app can use this ID to determine whether it recognizes the card. If an app recognizes the card, the app may be able to postpone certain operations that were completed previously. However the contents of the card may have changed since the card was last accessed by the app.

For example, consider an app that indexes ebooks. If the app has previously scanned the whole SD card for ebook files and created an index of the ebooks, it can display the list immediately if the card is reinserted and the app recognizes the card. Separately it can start a low-priority background thread to search for new ebooks. It can also handle a failure to find an ebook that existed previously when the user tries to access the deleted ebook.

The name of the property that contains this ID is **WindowsPhone.ExternalStorageId**.

C#

```
using Windows.Storage;

// Get the logical root folder for all external storage devices.
StorageFolder externalDevices =
Windows.Storage.KnownFolders.RemovableDevices;

// Get the first child folder, which represents the SD card.
StorageFolder sdCard = (await
externalDevices.GetFoldersAsync()).FirstOrDefault();

if (sdCard != null)
{
    var allProperties = sdCard.Properties;
    IEnumerable<string> propertiesToRetrieve = new List<string> {
"WindowsPhone.ExternalStorageId" };

    var storageIdProperties = await
allProperties.RetrievePropertiesAsync(propertiesToRetrieve);

    string cardId =
(string)storageIdProperties["WindowsPhone.ExternalStorageId"];

    if (...) // If cardID matches the cached ID of a recognized card.
    {
        // Card is recognized. Index contents opportunistically.
    }
    else
    {
        // Card is not recognized. Index contents immediately.
    }
}
```


File access permissions

Article • 10/20/2022

Universal Windows Platform (UWP) apps can access certain file system locations by default. Apps can also access additional locations through the file picker, or by declaring capabilities.

Locations that UWP apps can access

When you create a new app, you can access the following file system locations by default:

Application install directory

The folder where your app is installed on the user's system.

There are two primary ways to access files and folders in your app's install directory:

1. You can retrieve a [StorageFolder](#) that represents your app's install directory, like this:

```
C#  
  
Windows.Storage.StorageFolder installedLocation =  
    Windows.ApplicationModel.Package.Current.InstalledLocation;
```

You can then access files and folders in the directory using [StorageFolder](#) methods. In the example, this **StorageFolder** is stored in the `installDirectory` variable. You can learn more about working with your app package and install directory from the [App package information sample](#) on GitHub.

2. You can retrieve a file directly from your app's install directory by using an app URI, like this:

```
C#  
  
using Windows.Storage;  
StorageFile file = await StorageFile.GetFileFromApplicationUriAsync(new  
    Uri("ms-appx:///file.txt"));
```

When [GetFileFromApplicationUriAsync](#) completes, it returns a [StorageFile](#) that represents the `file.txt` file in the app's install directory (`file` in the example).

The "ms-appx:///\" prefix in the URI refers to the app's install directory. You can learn more about using app URIs in [How to use URIs to reference content](#).

In addition, and unlike other locations, you can also access files in your app install directory by using some [Win32 and COM for Universal Windows Platform \(UWP\) apps](#) and some [C/C++ Standard Library functions from Microsoft Visual Studio](#).

The app's install directory is a read-only location. You can't gain access to the install directory through the file picker.

Access application data locations

The folders where your app can store data. These folders (local, roaming and temporary) are created when your app is installed.

There are two primary ways to access files and folders from your app's data locations:

1. Use [ApplicationData](#) properties to retrieve an app data folder.

For example, you can use [ApplicationData.LocalFolder](#) to retrieve a [StorageFolder](#) that represents your app's local folder like this:

```
C#  
  
using Windows.Storage;  
StorageFolder localFolder = ApplicationData.Current.LocalFolder;
```

If you want to access your app's roaming or temporary folder, use the [RoamingFolder](#) or [TemporaryFolder](#) property instead.

After you retrieve a [StorageFolder](#) that represents an app data location, you can access files and folders in that location by using [StorageFolder](#) methods. In the example, these [StorageFolder](#) objects are stored in the `localFolder` variable. You can learn more about using app data locations from the guidance on the [ApplicationData class](#) page, and by downloading the [Application data sample](#)  from GitHub.

2. You can retrieve a file directly from your app's local folder by using an app URI, like this:

```
C#  
  
using Windows.Storage;  
StorageFile file = await StorageFile.GetFileFromApplicationUriAsync(new
```

```
Uri("ms-appdata:///local/file.txt"));
```

When [GetFileFromApplicationUriAsync](#) completes, it returns a [StorageFile](#) that represents the `file.txt` file in the app's local folder (`file` in the example).

The "ms-appdata:///local/" prefix in the URI refers to the app's local folder. To access files in the app's roaming or temporary folders use "ms-appdata:///roaming/" or "ms-appdata:///temporary/" instead. You can learn more about using app URIs in [How to load file resources](#).

In addition, and unlike other locations, you can also access files in your app data locations by using some [Win32 and COM for UWP apps](#) and some C/C++ Standard Library functions from Visual Studio.

You can't access the local, roaming, or temporary folders through the file picker.

Access removable devices

Additionally, your app can access some of the files on connected devices by default. This is an option if your app uses the [AutoPlay extension](#) to launch automatically when users connect a device, like a camera or USB thumb drive, to their system. The files your app can access are limited to specific file types that are specified via File Type Association declarations in your app manifest.

Of course, you can also gain access to files and folders on a removable device by calling the file picker (using [FileOpenPicker](#) and [FolderPicker](#)) and letting the user pick files and folders for your app to access. Learn how to use the file picker in [Open files and folders with a picker](#).

ⓘ Note

For more info about accessing an SD card or other removable devices, see [Access the SD card](#).

User's Downloads folder

The folder where downloaded files are saved by default.

By default, your app can only access files and folders in the user's Downloads folder that your app created. However, you can gain access to files and folders in the user's Downloads folder by calling a file picker ([FileOpenPicker](#) or [FolderPicker](#)) so that users can navigate and pick files or folders for your app to access.

- You can create a file in the user's Downloads folder like this:

```
C#  
  
using Windows.Storage;  
StorageFile newFile = await  
DownloadsFolder.CreateFileAsync("file.txt");
```

[DownloadsFolder.CreateFileAsync](#) is overloaded so that you can specify what the system should do if there is already an existing file in the Downloads folder that has the same name. When these methods complete, they return a [StorageFile](#) that represents the file that was created. This file is called `newFile` in the example.

- You can create a subfolder in the user's Downloads folder like this:

```
C#  
  
using Windows.Storage;  
StorageFolder newFolder = await DownloadsFolder.CreateFolderAsync("New  
Folder");
```

[DownloadsFolder.CreateFolderAsync](#) is overloaded so that you can specify what the system should do if there is already an existing subfolder in the Downloads folder that has the same name. When these methods complete, they return a [StorageFolder](#) that represents the subfolder that was created. This file is called `newFolder` in the example.

Accessing additional locations

In addition to the default locations, an app can access additional files and folders by [declaring capabilities in the app manifest](#) or by [calling a file picker](#) to let the user pick files and folders for the app to access.

Apps that declare the [AppExecutionAlias](#) extension have file-system permissions from the directory that they are launched from in the console window, and downwards.

Retaining access to files and folders

When your app retrieves a file or folder via a picker, a file activation, a drag-and-drop operation, etc. it only has access to that file or folder until the app is terminated. If you would like to automatically access the file or folder in the future, you can add it to the [FutureAccessList](#) so that your app can readily access that item in the future. You can also use the [MostRecentlyUsedList](#) to easily manage a list of recently-used files.

Capabilities for accessing other locations

The following table lists additional locations that you can access by declaring one or more capabilities and using the associated [Windows.Storage](#) API.

 Expand table

Location	Capability	Windows.Storage API
All files that the user has access to. For example: documents, pictures, photos, downloads, desktop, OneDrive, etc.	broadFileSystemAccess This is a restricted capability. Access is configurable in Settings > Privacy > File system . Because users can grant or deny the permission any time in Settings , you should ensure that your app is resilient to those changes. If you find that your app does not have access, you may choose to prompt the user to change the setting by providing a link to the Windows file system access and privacy article. Note that the user must close the app, toggle the setting, and restart the app. If they toggle the setting while the app is running, the platform will suspend your app so that you can save the state, then forcibly terminate the app in order to apply the new setting. In the April 2018 update, the default for the permission is On. In the October 2018 update, the default is Off. If you submit an app to the Store that declares this capability, you will need to supply additional descriptions of why your app needs this capability, and how it intends to use it. This capability works for APIs in the Windows.Storage namespace. See the Example section at the end of this article for an example of how to enable this capability in your app. Note: This capability is not supported on Xbox.	n/a

Location	Capability	Windows.Storage API
Documents	documentsLibrary Note: You must add File Type Associations to your app manifest that declare specific file types that your app can access in this location. Use this capability if your app: - Facilitates cross-platform offline access to specific OneDrive content using valid OneDrive URLs or Resource IDs - Saves open files to the user's OneDrive automatically while offline	KnownFolders.DocumentsLibrary
Music	musicLibrary Also see Files and folders in the Music, Pictures, and Videos libraries.	KnownFolders.MusicLibrary
Pictures	picturesLibrary Also see Files and folders in the Music, Pictures, and Videos libraries.	KnownFolders.PicturesLibrary
Videos	videosLibrary Also see Files and folders in the Music, Pictures, and Videos libraries.	KnownFolders.VideosLibrary
Removable devices	removableStorage Note You must add File Type Associations to your app manifest that declare specific file types that your app can access in this location. Also see Access the SD card.	KnownFolders.RemovableDevices
Homegroup libraries	At least one of the following capabilities is needed. - musicLibrary - picturesLibrary - videosLibrary	KnownFolders.HomeGroup
Media server devices (DLNA)	At least one of the following capabilities is needed. - musicLibrary - picturesLibrary - videosLibrary	KnownFolders.MediaServerDevices

Location	Capability	Windows.Storage API
Universal Naming Convention (UNC) folders	A combination of the following capabilities is needed. The home and work networks capability: - privateNetworkClientServer And at least one internet and public networks capability: - internetClient - internetClientServer And, if applicable, the domain credentials capability: - enterpriseAuthentication Note: You must add File Type Associations to your app manifest that declare specific file types that your app can access in this location.	Retrieve a folder using: StorageFolder.GetFolderFromPathAsync Retrieve a file using: StorageFile.GetFileFromPathAsync

Example

This example adds the restricted **broadFileSystemAccess** capability. In addition to specifying the capability, the `rescap` namespace must be added, and is also added to `IgnorableNamespaces`.

XAML

```
<Package
  ...

  xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows10/restrictedcapabilities"
  IgnorableNamespaces="uap mp rescap">
  ...
  <Capabilities>
    <rescap:Capability Name="broadFileSystemAccess" />
  </Capabilities>
```

ⓘ Note

For a complete list of app capabilities, see [App capability declarations](#).

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Fast access to file properties in UWP

Article • 10/20/2022

Learn how to quickly gather a list of files and their properties from a library and use those properties in an app.

Prerequisites

- **Asynchronous programming for Universal Windows Platform (UWP) apps** You can learn how to write asynchronous apps in C# or Visual Basic, see [Call asynchronous APIs in C# or Visual Basic](#). To learn how to write asynchronous apps in C++, see [Asynchronous programming in C++](#).
- **Access permissions to Libraries** The code in these examples requires the `picturesLibrary` capability, but your file location may require a different capability, or no capability at all. To learn more, see [File access permissions](#).
- **Simple file enumeration** This example uses [QueryOptions](#) to set a few advanced enumeration properties. To learn more about just getting a simple list of files for a smaller directory, see [Enumerate and query files and folders](#).

Usage

Many apps need to list the properties of a group of files, but don't always need to interact with the files directly. For example, a music app plays (opens) one file at a time, but it needs the properties of all of the files in a folder so the app can show the song queue, or so the user can choose a valid file to play.

The examples on this page shouldn't be used in apps that will modify the metadata of every file or apps that interact with all the resulting `StorageFiles` beyond reading their properties. See [Enumerate and query files and folders](#) for more information.

Enumerate all the pictures in a location

In this example, we will

- Build a [QueryOptions](#) object to specify that the app wants to enumerate the files as quickly as possible.
- Fetch file properties by paging `StorageFile` objects into the app. Paging the files in reduces the memory used by the app and improves its perceived responsiveness.

Creating the query

To build the query, we use a `QueryOptions` object to specify that the app is interested in enumerating only certain types of images files and to filter out files protected with Windows Information Protection (`System.Security.EncryptionOwners`).

It is important to set the properties the app is going to access using [`QueryOptions.SetPropertyPrefetch`](#). If the app accesses a property that isn't prefetched, it will incur a significant performance penalty.

Setting [`IndexerOption.OnlyUseIndexerAndOptimizeForIndexedProperties`](#) tells the system to return results as quickly as possible, but to only include the properties specified in [`SetPropertyPrefetch`](#).

Paging in the results

Users may have thousands or millions of files in their pictures library, so calling [`GetFilesAsync`](#) would overwhelm their machine because it creates a `StorageFile` for each image. This can be solved by creating a fixed number of `StorageFiles` at one time, processing them into the UI, and then releasing the memory.

In our example, we do this by using [`StorageFileQueryResult.GetFilesAsync\(UInt32 StartIndex, UInt32 maxNumberOfItems\)`](#) to only fetch 100 files at a time. The app will then process the files and allow the OS to release that memory afterwards. This technique caps the maximum memory of the app and ensures the system stays responsive. Of course, you will need to adjust the number of files returned for your scenario, but to ensure a responsive experience for all users, it's recommended to not fetch more than 500 files at one time.

Example

C#

```
StorageFolder folderToEnumerate = KnownFolders.PicturesLibrary;
// Check if the folder is indexed before doing anything.
IndexedState folderIndexedState = await
folderToEnumerate.GetIndexedStateAsync();
if (folderIndexedState == IndexedState.NotIndexed || folderIndexedState ==
IndexedState.Unknown)
{
    // Only possible in indexed directories.
    return;
}

QueryOptions picturesQuery = new QueryOptions()
{
    FolderDepth = FolderDepth.Deep,
    // Filter out all files that have WIP enabled
```

```

        ApplicationSearchFilter = "System.Security.EncryptionOwners:[]",
        IndexerOption =
IndexerOption.OnlyUseIndexerAndOptimizeForIndexedProperties
};

picturesQuery.FileTypeFilter.Add(".jpg");
string[] otherProperties = new string[]
{
    SystemProperties.GPS.LatitudeDecimal,
    SystemProperties.GPS.LongitudeDecimal
};

picturesQuery.SetPropertyPrefetch(PropertyPrefetchOptions.BasicProperties |
PropertyPrefetchOptions.ImageProperties,
                                otherProperties);
SortEntry sortOrder = new SortEntry()
{
    AscendingOrder = true,
    PropertyName = "System.FileName" // FileName property is used as an
example. Any property can be used here.
};
picturesQuery.SortOrder.Add(sortOrder);

// Create the query and get the results
uint index = 0;
const uint stepSize = 100;
if (!folderToEnumerate.AreQueryOptionsSupported(picturesQuery))
{
    log("Querying for a sort order is not supported in this location");
    picturesQuery.SortOrder.Clear();
}
StorageFileQueryResult queryResult =
folderToEnumerate.CreateFileQueryWithOptions(picturesQuery);
IReadOnlyList<StorageFile> images = await queryResult.GetFilesAsync(index,
stepSize);
while (images.Count != 0 || index < 10000)
{
    foreach (StorageFile file in images)
    {
        // With the OnlyUseIndexerAndOptimizeForIndexedProperties set, this
won't
        // be async. It will run synchronously.
        var imageProps = await file.Properties.GetImagePropertiesAsync();

        // Build the UI
        log(String.Format("{0} at {1}, {2}",
                        file.Path,
                        imageProps.Latitude,
                        imageProps.Longitude));
    }
    index += stepSize;
    images = await queryResult.GetFilesAsync(index, stepSize);
}

```

Results

The resulting StorageFile files only contain the properties requested, but are returned 10 times faster compared to the other IndexerOptions. The app can still request access to properties not already included in the query, but there is a performance penalty to open the file and retrieve those properties.

Adding folders to Libraries

Apps can request the user to add the location to the index using [StorageLibrary.RequestAddFolderAsync](#). Once the location is included, it will be automatically indexed and apps can use this technique to enumerate the files.

See also

[QueryOptions API Reference](#)

[Enumerate and query files and folders](#)

[File access permissions](#)

AI for Windows apps

Learn how to take advantage of state-of-the-art local models, NPU hardware, and remote APIs to make your apps smarter than ever before.

Introducing AI for Windows

GET STARTED

[FAQs about using AI on Windows](#)

[Get started using AI-backed APIs locally or in the cloud](#)

[Get started using ML models in your app](#)

[Responsible Generative AI Development on Windows](#)

[AI on Windows Sample Gallery](#)

[Our principles for using AI-generated content on Microsoft Learn](#)

Writing apps using local AI models

HOW-TO GUIDE

[Copilot+ PCs Developer Guide](#)

[Web Neural Network API \(WebNN\) Overview](#)

[Get started with WebNN with Windows](#)

[DirectML expands NPU support to Copilot+ PCs and WebNN](#) [↗](#)

[Find pre-built ML models](#)

[AI Toolkit for Visual Studio Code overview](#)

Windows Copilot Runtime

CONCEPT

[Overview of Windows Copilot Runtime](#)

[Phi Silica API - Small Language Model with reasoning and language understanding](#)

[Text Recognition API \(Optical Character Recognition / OCR\)](#)

[Recall API - Retrieval of past activities for user to continue where they left off](#)

[Imaging API - Image Super Resolution to scale and sharpen, Image Segmentation to identify objects.](#)

[Studio Effects - NPU-accelerated camera and microphone AI effects](#)

Other Windows AI tools

OVERVIEW

[Build cutting-edge, market-ready AI applications.](#) 

[Develop AI apps with .NET](#)

[Introducing Microsoft.Extensions.AI Preview](#) 

[Introduction to Semantic Kernel](#)

[OpenAI basic samples](#) 

DirectML for hardware-accelerated Machine Learning

OVERVIEW

[DirectML Overview](#)

[Get started with DirectML](#)

[DirectML Version History](#)

[DirectML developer tools](#)

[Sample applications](#)

DirectML Programming Guide

HOW-TO GUIDE

[Binding in DirectML](#)

[UAV barriers and resource state barriers](#)

[Resource lifetime and synchronization](#)

[Using strides to express padding, memory layout](#)

[Using fused operators to improve performance](#)

[Troubleshooting - using the debug layer](#)

[Troubleshooting - Handling errors and device removal](#)

DirectML support for PyTorch or WebNN

CONCEPT

[Enable PyTorch with DirectML on Windows](#)

[Enable PyTorch with DirectML on WSL 2](#)

[DirectML expands NPU support to Copilot+ PCs and WebNN](#) 

DirectML API Reference

REFERENCE

[DirectML interfaces](#)

[DirectML functions](#)

[DirectML structures](#)

[DirectML enumerations](#)

[DirectML constants](#)

Additional resources

CONCEPT

[Microsoft AI learning hub](#)

[Microsoft Copilot learning hub](#)

[Microsoft Responsible AI](#) 

[Microsoft AI Show Live](#)

Choose a UWP version

Article • 10/20/2022

Each version of Windows 10 and Windows 11 have brought new and improved features to the UWP platform. When creating a UWP app in Microsoft Visual Studio, you can choose which version to target. Projects using [.NET Standard 2.0](#) must have a **Minimum Version** of Build 16299 or later.

Warning

UWP projects created in current versions of Visual Studio cannot be opened in Visual Studio 2015.

The following table describes the available versions of Windows 10 and Windows 11. Please note that this table only applies for building UWP apps, which are only supported on Windows 10 and Windows 11. You cannot develop UWP apps for older versions of Windows, and you must have [installed the appropriate build of the SDK](#) in order to target that version.

 Expand table

Version	Description
Build 19041 (version 2004)	This is the latest version of Windows 10, released in May 2020. Highlighted features of this release include: * WSL2: Windows Subsystem for Linux has been updated with a new architectural model, and now runs an actual Linux kernel on Windows. Learn more at about WSL2. * MSIX: New features within Windows provide further support for the modern MSIX app packaging format, including the ability to create packages with included services, creation of hosted apps, and the ability to include features that require package identity in unpackaged apps. Learn more in the MSIX docs. For more information on these and the many other features added in this release of Windows, visit the Dev Center or the more in-depth page on What's new in Windows 10 for developers
Build 18362 (version 1903)	This version of Windows 10 was released in April 2019. Some highlighted features from this release include: * XAML Islands: Windows 10 now enables you to use UWP controls in non-UWP desktop applications. If you're developing for WPF, Windows Forms, or C++ Win32, check out how you can add the latest Windows 10 UI features to your existing app. * Windows Subsystem for Linux: You can now access Linux files directly from

Version	Description
	within Windows, and use several new command line options. See the latest at about WSL. For information on these and many other features added in this release of Windows, visit What's new in build 18362
Build 17763 (version 1809)	This version of Windows 10 was released in October 2018. Please note that you <i>must</i> be using Visual Studio 2017 or Visual Studio 2019 in order to target this version of Windows. Some highlighted features from this release include: * Windows Machine Learning: Windows Machine Learning has now officially launched, providing features like faster evaluation and support for cutting-edge machine learning models. To learn more about the platform, see Windows Machine Learning. * Fluent Design: New features such as menu bar, command bar flyout, and XAML property animations have been added to Windows 10. See the latest at the Fluent design overview. For information on these and many other features added in this release of Windows, visit What's new in build 17763
Build 17134 (version 1803)	This is version of Windows 10 was released in April 2018. Please note that you <i>must</i> be using Visual Studio 2017 or Visual Studio 2019 in order to target this version of Windows. Some highlighted features from this release include: * Fluent Design: New features such as tree view, pull-to-refresh, and navigation view have been added to Windows 10. See the latest at the Fluent design overview. * Console UWP apps: You can now write C++ /WinRT or /CX UWP console apps that run in a console window such as a DOS or PowerShell console window. For information on these and many other features added in this release of windows, visit What's new in build 17134
Build 16299 (Fall Creators Update, version 1709)	This version of Windows 10 was released in October 2017. Please note that you <i>must</i> be using Visual Studio 2017 or Visual Studio 2019 in order to target this version of Windows. Some highlighted features from this release include: * .NET Standard 2.0: Enjoy a massive increase in the number of .NET APIs and incorporate your favorite NuGet packages and third party libraries into .NET Standard. See more details and explore the documentation here. Please note that you must set your minimum version to Build 16299 to access these new APIs. * Fluent Design: Use light, depth, perspective, and movement to enhance your app and help users focus on important UI elements. * Conditional XAML: Easily set properties and instantiate objects based on the presence of an API at runtime, enabling your apps to run seamlessly across devices and versions. For information on these and many other features added in this release of windows, visit What's new in Windows 10 for developers

Version	Description
Build 15063 (Creators Update, version 1703)	This version of Windows 10 was released in March 2017. Please note that you <i>must</i> be using Visual Studio 2017 or Visual Studio 2019 in order to target this version of Windows. Some highlighted features from this release include: * Ink Analysis: Windows Ink can now categorize ink strokes into either writing or drawing strokes, and recognized text, shapes, and basid layout structures. * Windows.Ui.Composition APIs: Easily combine and apply animations across your app. * Live Editing: Edit XAML while your app is running, and see the changes applied in real-time. For information on these and many other features added in this release of windows, visit What's new in build 15063
Build 14393 (Anniversary Update, version 1607)	This version of Windows 10 was released in July 2016. Some highlighted features from this release include: * Windows Ink: New InkCanvas and InkToolbar controls. * Cortana APIs: Use new Cortana Actions to integrate Cortana support with specific functions of your app. * Windows Hello: Microsoft Edge now supports Windows Hello, giving web developers access to biometric authentication. For information on these and many other features added in this release of windows, visit What's new in build 14393
Build 10586 (November Update, version 1511)	This version of Windows 10 was released in November 2015. Highlighted features include the introduction of ORTC (object real-time communications) APIs for video communication in Microsoft Edge and Providers APIs to enable apps to use Windows Hello face authentication. More information on features introduced in this build.
Build 10240 (Windows 10, version 1507)	This is the initial release version of Windows 10, from July 2015. More information on features introduced in this build.

We highly recommend that new developers and developers writing code for a general audience always use the latest build of Windows (19041). Developers writing Enterprise apps should strongly consider supporting an older **Minimum Version**.

What's different in each UWP version?

New and changed APIs for UWP are available in every successive version of Windows 10 and Windows 11. For specific information about what features were added in which version, see [What's new for developers in Windows 10/11](#).

For reference topics that enumerate all device families and their versions, and all API contracts and their versions, see [Device families](#) and [API contracts](#).

.NET API availability in UWP versions

UWP supports a limited subset of .NET APIs, which are available regardless of the **Target Version** or **Minimum Version** of your project. [This page provides more information on the types available](#).

If you wish to create reusable cross-platform libraries, .NET Standard is supported on UWP. The [.NET Standard documentation](#) provides information on which .NET Standard is supported in which UWP versions.

If you are developing a Desktop app, see instead [.NET Framework versions and dependencies](#) for detailed information on .NET framework availability.

Choose which version to use for your app

In the **New Universal Windows Project** dialog in Visual Studio, you can choose a version for **Target Version** and for **Minimum Version**. Additionally, you can change the **Target Version** and **Minimum Version** of your UWP app in the *application* section of the app's **Properties**.

- **Target Version.** The version of Windows 10 or Windows 11 that your app is intended to run on. This sets the *TargetPlatformVersion* setting in your project file. It also determines the value of the *TargetDeviceFamily@MaxVersionTested* attribute in your app package manifest. The value you choose specifies the version of the UWP platform that your project is targeting—and therefore the set of APIs available to your app—so we recommend that you choose the most recent version possible. For more info about your app package manifest, and some guidelines around configuring *TargetDeviceFamily* manually, see [TargetDeviceFamily](#).
- **Minimum Version.** The earliest version of Windows 10 or Windows 11 needed to support the basic functions of your app. This sets the *TargetPlatformMinVersion* setting in your project file. It also determines the value of the *TargetDeviceFamily@MinVersion* attribute in your app package manifest. The value you choose specifies the minimum version of the UWP platform that your project can work with.

Be aware that you're declaring that your app works on any version of Windows in the range from **Minimum Version** to **Target Version**. If those two are the same version then

you don't need to do anything special. If they're different, then here are some things to be aware of.

- In your code, you can freely (that is, without conditional checks) call any API that exists in the version specified by **Minimum Version**.
- Ensure that you test your code on a device running the **Minimum Version**, to be sure that it works without requiring APIs only present in the **Target Version**.
- The value of **Target Version** is used to identify all the references (contract winmds) used to compile your project. But those references will enable you to compile your code with calls to APIs that won't necessarily exist on devices that you've declared that you support (via **Minimum Version**). Therefore, any API that was introduced after **Minimum Version** will need to be called via adaptive code. For more information about adaptive code, see [Version adaptive code](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Debugging, testing, and performance

Article • 10/20/2022

This section shows you how to use Microsoft Visual Studio to debug, test, and optimize your app. It also includes tools like the Windows Device Portal (for device monitoring and configuration) and the Windows App Certification Kit (to prepare your app for the Microsoft Store).

Topic	Description
Deploying and debugging UWP apps	This article guides you through the steps to target various deployment and debugging targets.
Testing and debugging tools for Process Lifetime Management (PLM)	Tools and techniques for debugging and testing how your app works with Process Lifetime Management.
Test with the Microsoft Emulator for Windows 10 Mobile	Simulate real-world interaction with a device and test the features of your app by using the tools included with Microsoft Emulator for Windows 10 Mobile. The emulator is a desktop application that emulates a mobile device running Windows 10. It provides a virtualized environment in which you can debug and test Windows apps without a physical device. It also provides an isolated environment for your application prototypes.
Test Surface Hub apps using Visual Studio	The Visual Studio simulator provides an environment where you can design, develop, debug, and test Universal Windows Platform (UWP) apps, including apps that you have built for Microsoft Surface Hub. The simulator does not use the same user interface as Surface Hub, but it is useful for testing how your app looks and behaves at the Surface Hub's screen size and resolution.
Deploy an app through loose file registration	This guide shows how to use the loose file layout to validate and share Windows 10 apps without needing to package them.
Beta testing	Beta testing gives you the chance to improve your app based on feedback from individuals outside of your app-development team who try your unreleased app on their own devices.
Windows Device Portal	The Windows Device Portal lets you configure and manage your device remotely over a network or USB connection.

Topic	Description
Windows App Certification Kit	To give your app the best chance of being published on the Microsoft Store, or becoming Windows Certified, validate and test it locally before you submit it for certification. This topic shows you how to install and run the Windows App Certification Kit.
Performance	Users expect their apps to remain responsive, to feel natural, and not to drain their battery. Technically, performance is a non-functional requirement but treating performance as a feature will help you deliver on your users' expectations. Specifying goals, and measuring, are key factors. Determine what your performance-critical scenarios are; define what good performance mean. Then measure early and often enough throughout the lifecycle of your project to be confident you'll hit your goals.
Version adaptive apps	Take advantage of the latest APIs and features while still reaching the broadest possible audience. Use runtime API checks to adapt your code and XAML at runtime to the features available on the version of Windows 10 where your app is running.

Deploying and debugging UWP apps

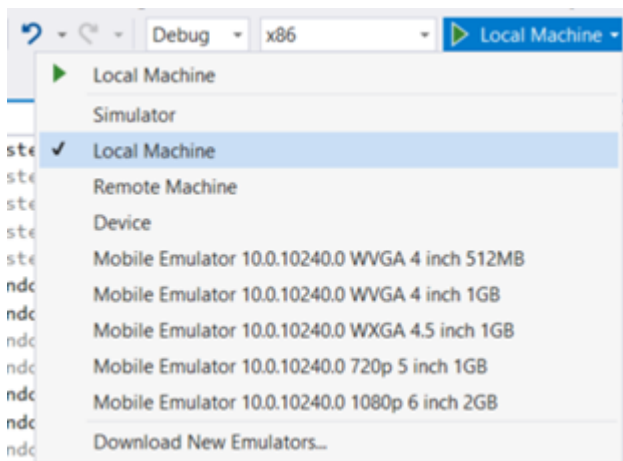
Article • 10/20/2022

This article guides you through the steps to target various deployment and debugging targets.

Microsoft Visual Studio allows you to deploy and debug your Universal Windows Platform (UWP) apps on a variety of Windows 10 devices. Visual Studio will handle the process of building and registering the app on the target device.

Picking a deployment target

To pick a target, go to the debug target drop-down next to the **Start Debugging** button and choose which target you want to deploy your app to. After the target is selected, select **Start Debugging (F5)** to deploy and debug on that target, or select **Ctrl+F5** to just deploy to that target.



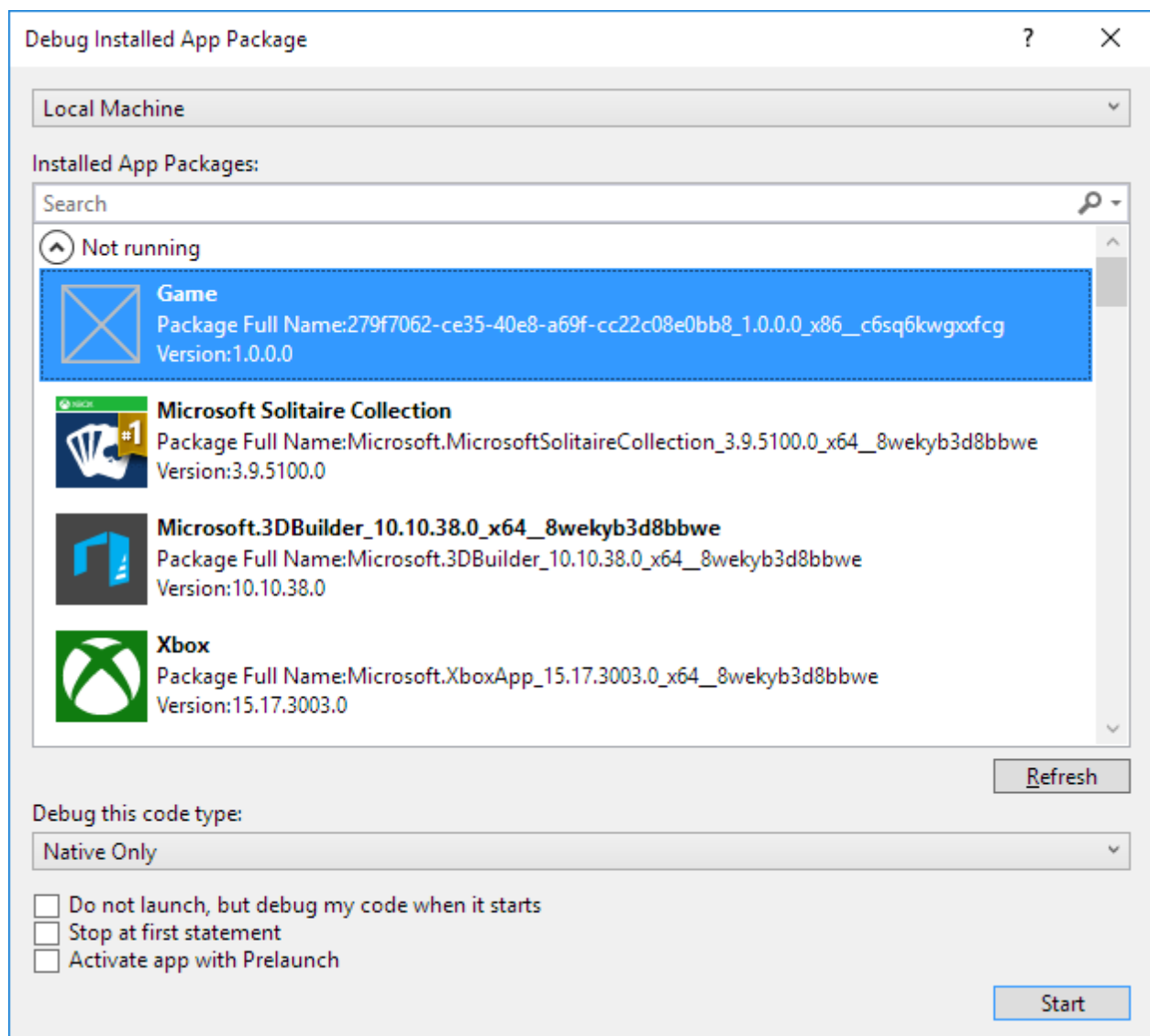
- **Simulator** will deploy the app to a simulated environment on your current development machine. This option is only available if your app's **Target Platform Min. Version** is less than or equal to the operating system on your development machine.
- **Local Machine** will deploy the app to your current development machine. This option is only available if your app's **Target Platform Min. Version** is less than or equal to the operating system on your development machine.
- **Remote Machine** will let you specify a remote target to deploy the app. More information about deploying to a remote machine can be found in [Specifying a remote device](#).
- **Device** will deploy the app to a USB connected device. The device must be developer unlocked and have the screen unlocked.

- An **Emulator** target will boot up and deploy the app to an emulator with the configuration specified in the name. Emulators are only available on Hyper-V enabled machines running Windows 8.1 or beyond.

Debugging deployed apps

Visual Studio can also attach to any running UWP app process by selecting **Debug**, and then **Attach to Process**. Attaching to a running process doesn't require the original Visual Studio project, but loading the process's [symbols](#) will help significantly when debugging a process that you don't have the original code for.

In addition, any installed app package can be attached and debugged by selecting **Debug**, **Other**, and then **Debug Installed App Packages**.



Selecting **Do not launch, but debug my code when it starts** will cause the Visual Studio debugger to attach to your UWP app when you launch it at a custom time. This is an effective way to debug control paths from [different launch methods](#), such as protocol activation with custom parameters.

UWP apps can be developed and compiled on Windows 8.1 or later, but require Windows 10 to run. If you are developing a UWP app on a Windows 8.1 PC, you can remotely debug a UWP app running on another Windows 10 device, provided that both the host and target computer are on the same LAN. To do this, download and install the [Remote Tools for Visual Studio](#) on both machines. The installed version must match the existing version of Visual Studio that you have installed, and the architecture you select (x86, x64) must also match that of your target app.

Package layout

As of Visual Studio 2015 Update 3, we have added the option for developers to specify the layout path for their UWP apps. This determines where the package layout is copied to on disk when you build your app. By default, this property is set relative to the project's root directory. If you do not modify this property, the behavior will remain the same as it has for previous versions of Visual Studio.

This property can be modified in the project's **Debug** properties.

If you want to include all layout files in your package when you create a package for your app, you must add the project property

```
<IncludeLayoutFilesInPackage>true</IncludeLayoutFilesInPackage>.
```

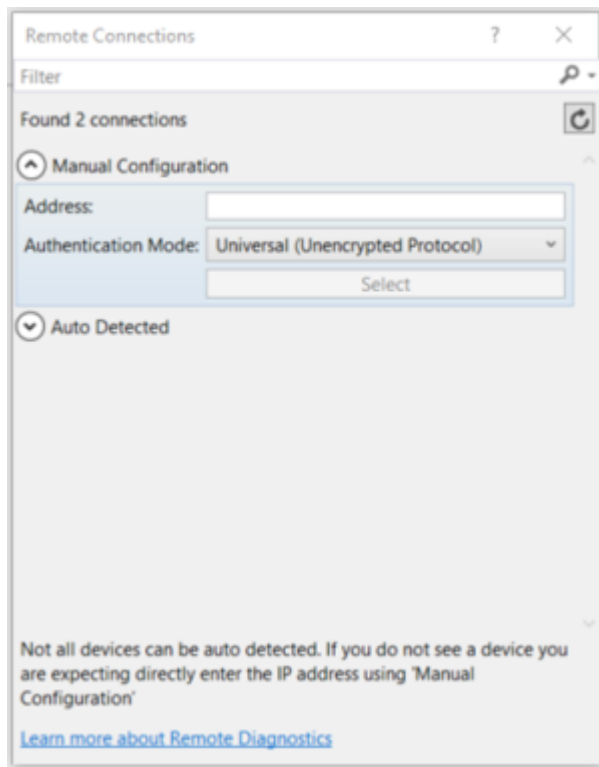
To add this property:

1. Right-click the project, and then select **Unload Project**.
2. Right-click the project, and then select **Edit [projectname].xxproj** (.xxproj will change depending on project language).
3. Add the property, and then reload the project.

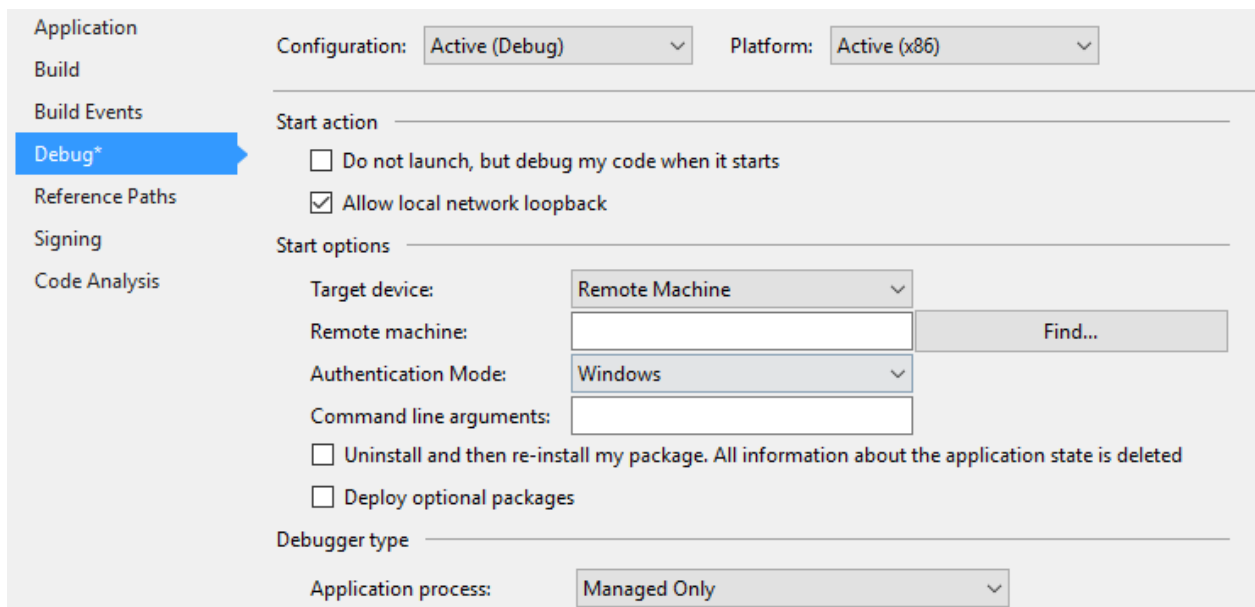
Specifying a remote device

C# and Microsoft Visual Basic

To specify a remote machine for C# or Microsoft Visual Basic apps, select **Remote Machine** in the debug target drop-down. The **Remote Connections** dialog will appear, which will let you specify an IP address or select a discovered device. By default, the **Universal** authentication mode is selected. To determine which authentication mode to use, see [Authentication modes](#).



To return to this dialog, you can open project properties and go to the **Debug** tab. From there, select **Find** next to **Remote machine**:



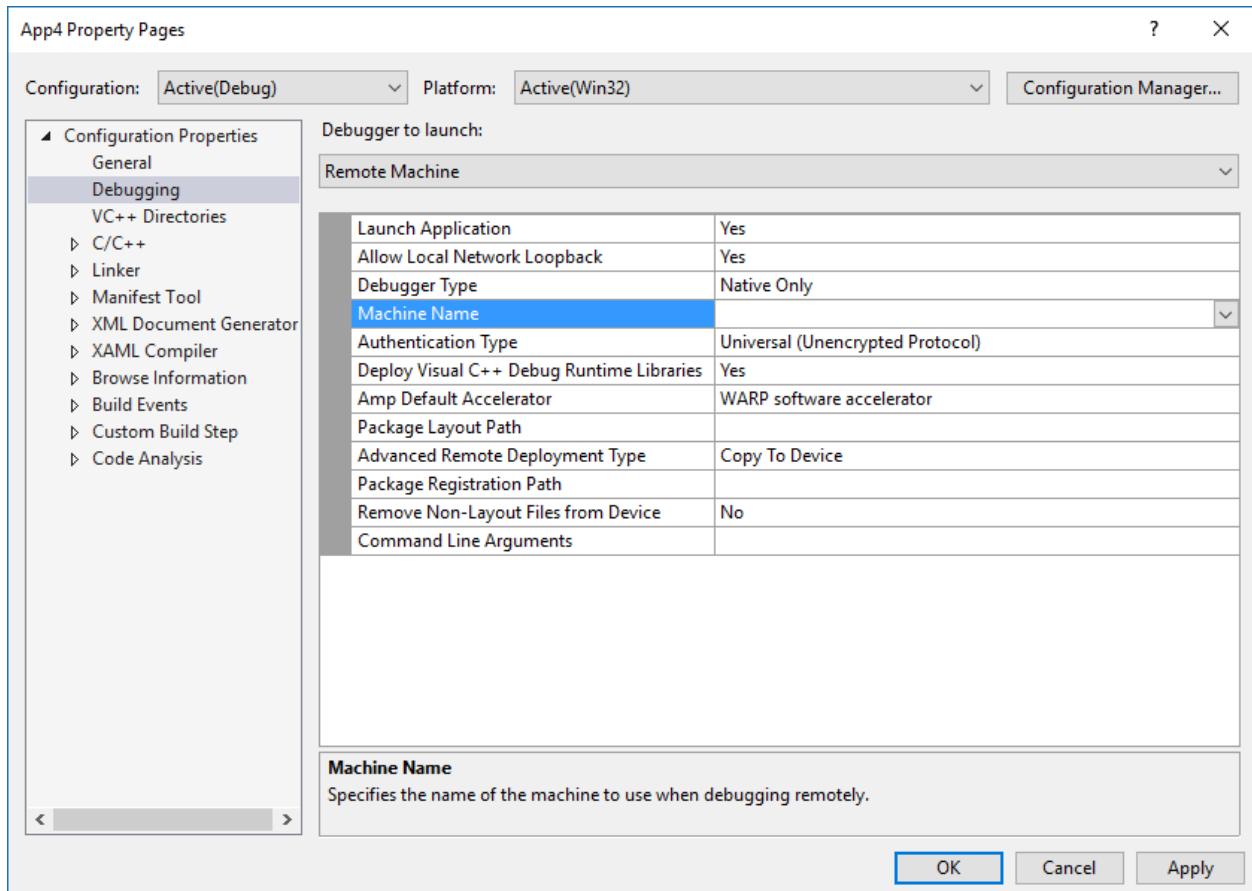
To deploy an app to a pre-Creators Update remote PC, you will also need to download and install the Visual Studio Remote Tools on the target PC. For full instructions, see [Remote PC instructions](#). However, as of the Creators Update PC also supports remote deployment.

C++ and JavaScript

To specify a remote machine target for a C++ or JavaScript UWP app:

1. In the **Solution Explorer**, right-click the project, and then click **Properties**.

2. Go to **Debugging** settings, and under **Debugger to launch**, select **Remote Machine**.
3. Enter the **Machine Name** (or click **Locate** to find one), and then set the **Authentication Type** property.



After the machine is specified, you can select **Remote Machine** in the debug target drop-down to return to that specified machine. Only one remote machine can be selected at a time.

Remote PC instructions

ⓘ Note

These instructions are only required for older versions of Windows 10. As of the Creators Update, a PC can be treated like an Xbox. That is, by enabling Device Discovery in the PC's Developer Mode menu and using Universal Authentication to PIN pair and connect with the PC.

To deploy to a pre-Creators Update remote PC, the target PC must have the Visual Studio Remote Tools installed. The remote PC must also be running a version of Windows that is greater than or equal to your apps **Target Platform Min. Version**

property. After you have installed the remote tools, you must launch the remote debugger on the target PC.

To do this, search for **Remote Debugger** in the **Start** menu, open it, and if prompted, allow the debugger to configure your firewall settings. By default, the debugger launches with Windows authentication. This will require user credentials if the signed-in user is not the same on both PCs.

To change it to **no authentication**, in the **Remote Debugger**, go to **Tools -> Options**, and then set it to **No Authentication**. After the remote debugger is set up, you must also ensure that you have set the host device to [Developer Mode](#). After that, you can deploy from your development machine.

For more information, see the [Visual studio Download Center](#) [page](#).

Passing command line debug arguments

In Visual Studio 2019, you can pass command line debug arguments when you start debugging UWP applications. You can access the command line debug arguments from the *args* parameter in the **OnLaunched** method of the [Application](#) class. To specify command line debug arguments, open the project's properties and navigate to the **Debug** tab.

Note

This is available in Visual Studio 2017 (version 15.1) for C#, VB, and C++. JavaScript is available in later versions. Command line debug arguments are available for all deployment types except for the Simulator.

For C# and VB UWP projects, you will see a **Command line arguments:** field under **Start options**.

Application Build

Build Events

Debug

Reference Paths

Signing

Code Analysis

Configuration: Active (Debug) Platform: Active (x86)

Start action

☐ Do not launch, but debug my code when it starts

☒ Allow local network loopback

Start options

Target device: Local Machine

Remote machine: Find...

Authentication Mode: Windows

Command line arguments: foo

☐ Uninstall and then re-install my package. All information about the application state is deleted

☐ Deploy optional packages

For C++ and JS UWP projects, you will see **Command Line Arguments** as a field in the **Debugging Properties**.

App4 Property Pages

Configuration: Active(Debug) Platform: Active(Win32) Configuration Manager...

Debugger to launch: Local Machine

Launch Application	Yes
Allow Local Network Loopback	Yes
Debugger Type	Native Only
Amp Default Accelerator	WARP software accelerator
Package Layout Path	
Command Line Arguments	

Once you specify the command line arguments, you can access the value of the argument in the App's **OnLaunched** method. The **LaunchActivatedEventArgs** object *args* will have an **Arguments** property with the value set to the text in the **Command Line Arguments** field.

```

/// <summary>
/// Invoked when the application is launched normally by the user
/// will be used such as when the application is launched from a taskbar
/// </summary>
/// <param name="e">Details about the launch process and user input</param>
protected override void OnLaunched(LaunchActivatedEventArgs e)
{
    Frame rootFrame = Window.Current.Content as Frame;
}

```

Authentication modes

There are three authentication modes for remote machine deployment:

- **Universal (Unencrypted Protocol):** Use this authentication mode whenever you are deploying to a remote device. Currently, this is for IoT devices, Xbox devices, and HoloLens devices, as well as PCs with Windows Version 1703 (Creators Update) or later. Universal (Unencrypted Protocol) should only be used on trusted networks. The debugging connection is vulnerable to malicious users who could intercept and change data being passed between the development and remote machine.
- **Windows:** This authentication mode is only intended to be used for a remote PC (desktop or laptop) running the Visual Studio Remote Tools. Use this authentication mode when you have access to the credentials of the signed-in user of the target machine. This is the most secure channel for remote deployment.
- **None:** This authentication mode is only intended to be used for a remote PC (desktop or laptop) running the Visual Studio Remote Tools. Use this authentication mode when you have a test machine set up in an environment that has a test account signed in and you cannot enter the credentials. Ensure that the remote debugger settings are set to accept no authentication.

Advanced remote deployment options

As of the release of Visual Studio 2015 Update 3, and the Windows 10 Anniversary Update, there are new advanced remote deployment options for certain Windows 10 devices. The advanced remote deployment options can be found on the **Debug** menu for project properties.

The new properties include:

- Deployment type
- Package registration path
- Keep all files on device – even those that are no longer a part of your layout

Requirements

To utilize the advanced remote deployment options, you must satisfy the following requirements:

- Have Visual Studio 2015 Update 3 or some later Visual Studio release installed with Windows 10 Tools 1.4.1 or later(which includes the Windows 10 Anniversary Update SDK) We recommend that you use the latest version of Visual Studio with updates to ensure you get all the newest development and security features.
- Target a Windows 10 Anniversary Update Xbox remote device or Windows 10 Creators Update PC
- Use Universal Authentication mode

Properties pages

For a C# or Visual Basic UWP app, the properties page will look like the following.

The screenshot shows the 'Start options' section with 'Target device' set to 'Remote Machine', a text field for 'Remote machine', and 'Authentication Mode' set to 'Universal (Unencrypted Protocol)'. There is a checkbox for 'Uninstall and then re-install my package'. The 'Debugger type' section has 'Application process' and 'Background task process' both set to 'Managed Only'. The 'Package layout' section has a text field for 'Layout folder path' set to 'c:\users\dajaco\documents\visual studio 2015' and a 'Browse...' button. The 'Advanced remote deployment options' section has 'Deployment type' set to 'Copy files to device', a text field for 'Package registration path' set to '(optional)', and a checkbox for 'Keep all files on device'.

For a C++ UWP app, the properties page will look like the following.

The screenshot shows the 'RemoteDeployCPP Property Pages' dialog box. The 'Configuration' is 'Active(Debug)' and the 'Platform' is 'Active(Win32)'. The 'Debugger to launch' is 'Remote Machine'. The 'Advanced Remote Deployment Type' is set to 'Copy To Device'. The 'Package Registration Path' is empty. The 'Remove Non-Layout Files from Device' is set to 'No'. The 'Advanced Remote Deployment Type' section has a description: 'Specifies the type of Remote Deployment - Choose 'Copy To Device' to physically copy the package layout to re...'. The 'OK', 'Cancel', and 'Apply' buttons are at the bottom.

Property	Value
Launch Application	Yes
Allow Local Network Loopback	Yes
Debugger Type	Native Only
Machine Name	
Authentication Type	Universal (Unencrypted Protocol)
Deploy Visual C++ Debug Runtime Libraries	Yes
Amp Default Accelerator	WARP software accelerator
Package Layout Path	
Advanced Remote Deployment Type	Copy To Device
Package Registration Path	
Remove Non-Layout Files from Device	No

Copy files to device

Copy files to device will physically transfer the files over the network to the remote device. It will copy and register the package layout that is built to the **Layout folder path**. Visual Studio will keep the files that are copied to the device in sync with the files in your Visual Studio project; however, there is an option to **keep all files on device – even those that are no longer a part of your layout**. Selecting this option means that any files that were previously copied to the remote device, but are no longer a part of your project, will remain on the remote device.

The **package registration path** specified when you **copy files to device** is the physical location on the remote device where the files are copied. This path can be specified as any relative path. The location where the files are deployed will be relative to a development files root that will vary depending on the target device. Specifying this path is useful for multiple developers sharing the same device and working on packages with some build variance.

Note

Copy files to device is currently supported on Xbox running Windows 10 Anniversary Update and PCs running Windows 10 Creators Update .

On the remote device, the layout gets copied to the following default location: `\\MY-DEVKIT\DevelopmentFiles\PACKAGE-REGISTRATION-PATH`

Register layout from network

When you choose to register the layout from the network, you can build your package layout to a network share and then register the layout on the remote device directly from the network. This requires that you specify a layout folder path (a network share) that is accessible from the remote device. The **Layout folder path** property is the path set relative to the PC running Visual Studio, while the **Package registration path** property is the same path, but specified relative to the remote device.

To successfully register the layout from the network, you must first make **Layout folder path** a shared network folder. To do this, right-click the folder in File Explorer, select **Share with > Specific people**, and then choose the users you would like to share the folder with. When you try to register the layout from the network, you will be prompted for credentials to ensure that you are registering as a user with access to the share.

For help with this, see the following examples:

- Example 1 (local layout folder, accessible as a network share):
 - **Layout folder path** = `D:\Layouts\App1`
 - **Package registration path** = `\\NETWORK-SHARE\Layouts\App1`
- Example 2 (network layout folder):
 - **Layout folder path** = `\\NETWORK-SHARE\Layouts\App1`
 - **Package registration path** = `\\NETWORK-SHARE\Layouts\App1`

When you first register the layout from the network, your credentials will be cached on the target device so you do not need to repeatedly sign in. To remove cached credentials, you can use the [WinAppDeployCmd.exe](#) tool from the Windows 10 SDK with the **deletecreds** command.

You cannot select **keep all files on device** when you register the layout from the network because no files are physically copied to the remote device.

ⓘ Note

Register layout from network is currently supported on Xbox running Windows 10 Anniversary Update and PCs running Windows 10 Creators Update.

On the remote device, the layout gets registered to the following default location depending on the device family: Xbox: `\\MY-DEVKIT\DevelopmentFiles\XrfsFiles` - this is a symlink to the **package registration path** PC does not use a symlink and instead directly registers the **package registration path**

Debugging options

On Windows 10, the startup performance of UWP apps is improved by proactively launching and then suspending apps in a technique called [prelaunch](#). Many apps will not need to do anything special to work in this mode, but some apps may need to adjust their behavior. To help debug any issues in these code paths, you can start debugging the app from Visual Studio in prelaunch mode.

Debugging is supported both from a Visual Studio project (**Debug -> Other Debug Targets -> Debug Universal Windows App Prelaunch**), and for apps already installed on the machine (**Debug -> Other Debug Targets -> Debug Installed App Package** by selecting the **Activate app with Prelaunch** check box). For more information, see [Debug UWP Prelaunch](#) [↗](#).

You can set the following deployment options on the **Debug** property page of the startup project:

- **Allow local network loopback**

For security reasons, a UWP app that is installed in the standard manner is not allowed to make network calls to the device it is installed on. By default, Visual Studio deployment creates an exemption from this rule for the deployed app. This exemption allows you to test communication procedures on a single machine. Before submitting your app to the Microsoft Store, you should test your app without the exemption.

To remove the network loopback exemption from the app:

- On the C# and Visual Basic **Debug** property page, clear the **Allow local network loopback** check box.
- On the JavaScript and C++ **Debugging** property page, set the **Allow Local Network Loopback** value to **No**.

- **Do not launch, but debug my code when it starts / Launch Application**

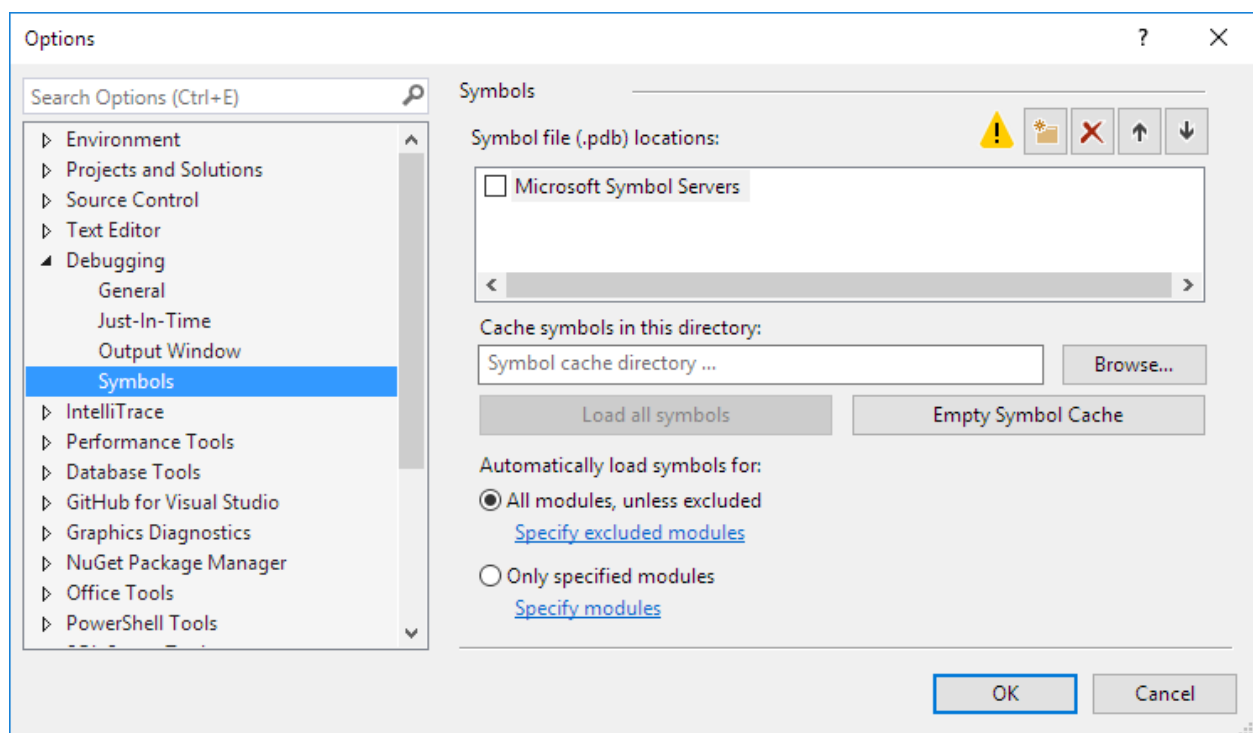
To configure the deployment to automatically start a debugging session when the app is launched:

- On the C# and Visual Basic **Debug** property page, select the **Do not launch, but debug my code when it starts** check box.
- On the JavaScript and C++ **Debugging** property page, set the **Launch Application** value to **Yes**.

Symbols

Symbol files contain a variety of very useful data when debugging code, such as variables, function names, and entry point addresses, allowing you to better understand exceptions and callstack execution order. Symbols for most variants of Windows are available through the [Microsoft Symbol Server](#) or can be downloaded for faster, offline lookups at [Download Windows Symbol Packages](#).

To set symbol options for Visual Studio, select **Tools > Options**, and then go to **Debugging > Symbols** in the dialog window.



To load symbols in a debugging session with [WinDbg](#), set the **sympath** variable to the symbol package location. For example, running the following command will load symbols from the Microsoft Symbol Server, and then cache them in the C:\Symbols directory:

Windows Command Prompt

```
.sympath SRV*C:\Symbols*http://msdl.microsoft.com/download/symbols
.reload
```

You can add more paths by using the `;` delimiter, or use the `.sympath+` command. For more advanced symbol operations that use WinDbg, see [Public and Private Symbols](#).

WinDbg

WinDbg is a powerful debugger that is shipped as part of the Debugging Tools for Windows suite, which is included in the [Windows SDK](#). The Windows SDK installation allows you to install Debugging Tools for Windows as a standalone product. While highly useful for debugging native code, we don't recommend WinDbg for apps written in managed code or HTML5.

To use WinDbg with UWP apps, you will need to first disable Process Lifetime Management (PLM) for your app package by using PLMDebug, as described in [Testing and debugging tools for Process Lifetime Management \(PLM\)](#).

Windows Command Prompt